

Trabajo Práctico 2: La fuga de la mazmorra

Trabajo práctico grupal · 6 integrantes · 8 semanas

Objetivo

El objetivo de este trabajo práctico es:

- Construir un **juego completo** aplicando todo lo visto en la materia.
- Usar **todas las estructuras de datos** del curso donde cada una hace falta de verdad: pila, cola, lista, cola de prioridad, tabla hash, árbol binario de búsqueda y grafo.
- Programar **100% orientado a objetos**: 5 tipos de monstruo y 10 tipos de cofre por polimorfismo.
- Generar los tableros **al azar** con un **generador de laberintos** configurado desde JSON, diseñado contra una interfaz para que el generador de **otro grupo** encaje sin cambiar una línea.
- Leer y escribir **archivos**: configuración y partidas en JSON, ranking, y la salida gráfica en **BMP**.
- Trabajar **en equipo** durante dos meses con Git, roles e hitos.

El juego

Una **mazmorra** es un conjunto de **planos**: grillas de filas × columnas donde cada celda es piso, pared, agua, lava, cofre, escalera, pasillo o salida. Adentro se mueven el **héroe** y los **monstruos**. El héroe avanza **de a pasos**: cada acción suya es un paso, y después de cada paso se mueven los monstruos. Al iniciar, el jugador elige una **dificultad**, y los planos **se generan al azar** con el generador de laberintos, según un archivo de configuración.

Un plano de ejemplo

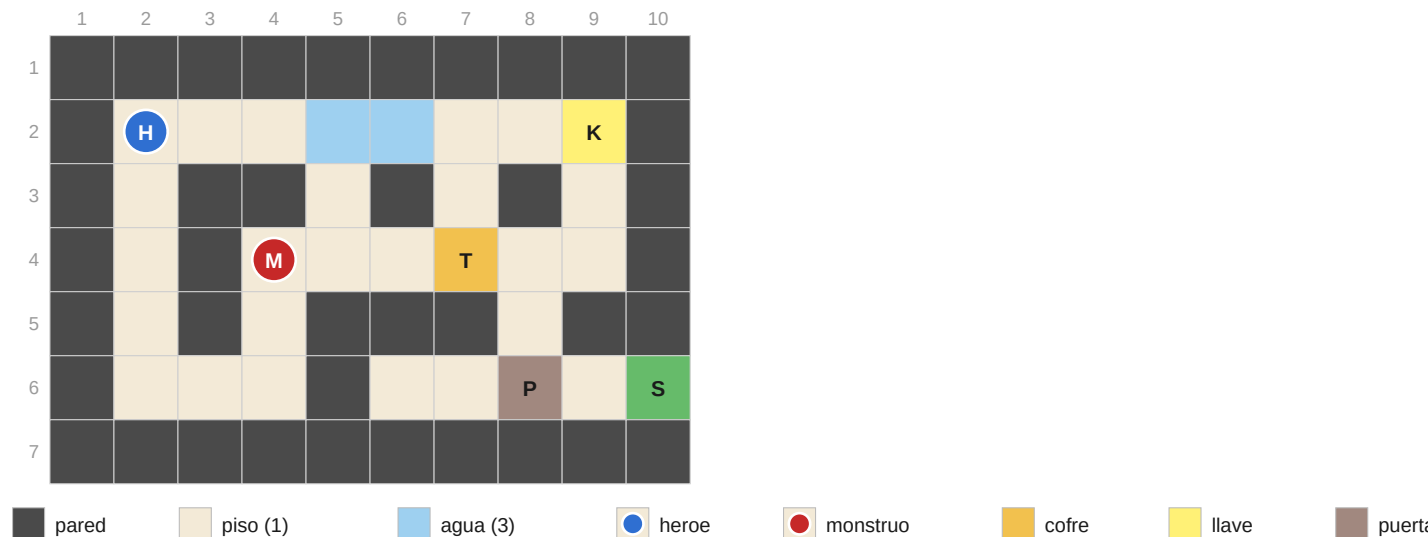


Figura 1. Un plano chico de 7 filas por 10 columnas, con el costo de energía de cada terreno entre paréntesis. Los planos del juego son más grandes y los produce el generador.

Las reglas

- 1) El héroe empieza con la **energía inicial** de la configuración (100 por defecto) y el inventario vacío.
- 2) En cada paso hace **una** acción: **moverse** una celda (arriba, abajo, izquierda o derecha), **usar** un objeto del inventario, o **esperar**.
- 3) Cada movimiento cuesta energía según el terreno al que entra: **piso 1, agua 3, lava 8**, configurables. Una pared no se puede pisar: no es una acción válida y no cuenta como paso.

- 4) Pisar un **cofre** lo abre: su contenido va al inventario o actúa de inmediato, según el tipo. Un cofre abierto suma 100 puntos y desaparece.
- 5) Pisar una **escalera** lleva al héroe al piso de arriba o de abajo del mismo tablero. Pisar un **pasillo** lo lleva a otro tablero. En los dos casos aparece en la celda de destino de la conexión. Los monstruos **nunca** cambian de plano.
- 6) Los **monstruos** tienen comportamientos distintos según su tipo. Los dos básicos: los **errantes** se mueven al azar; los **cazadores** avanzan una celda hacia el héroe **por el camino más corto**, respetando el costo del terreno. Si un monstruo alcanza al héroe hace lo que su tipo indica; el básico descuenta 30 de energía y vuelve a su posición inicial.
- 7) Hay una **agenda**: cosas que pasan en un paso futuro. Efectos de cofres que vencen, trampas que se rearmen, monstruos que despiertan. Al empezar cada paso se ejecutan los eventos cuyo momento llegó, en orden.
- 8) Los movimientos **no se deshacen**. Lo que se pisó, se pisó.
- 9) El juego se **gana** al pisar la salida, que está en un solo plano. Se **pierde** si la energía llega a 0. Puntaje: **cofres abiertos × 100 + energía restante × 2 - pasos**.
- 10) Una partida se puede **guardar** en cualquier momento y **retomar** después exactamente igual: todos los planos, el plano actual, héroe, inventario, efectos activos, monstruos y agenda.
- 11) Al terminar, el puntaje entra al **ranking**, que se conserva entre ejecuciones y muestra el top 10 con la dificultad.

Sobre el ritmo. Con entrada por consola, el juego avanza naturalmente de a pasos. Un grupo que use la ventana gráfica puede, si quiere, mover a los monstruos **por tiempo** con un temporizador, y contar la agenda en milisegundos en vez de pasos. Las reglas son las mismas; sólo cambia quién marca el ritmo.

Las tres dificultades

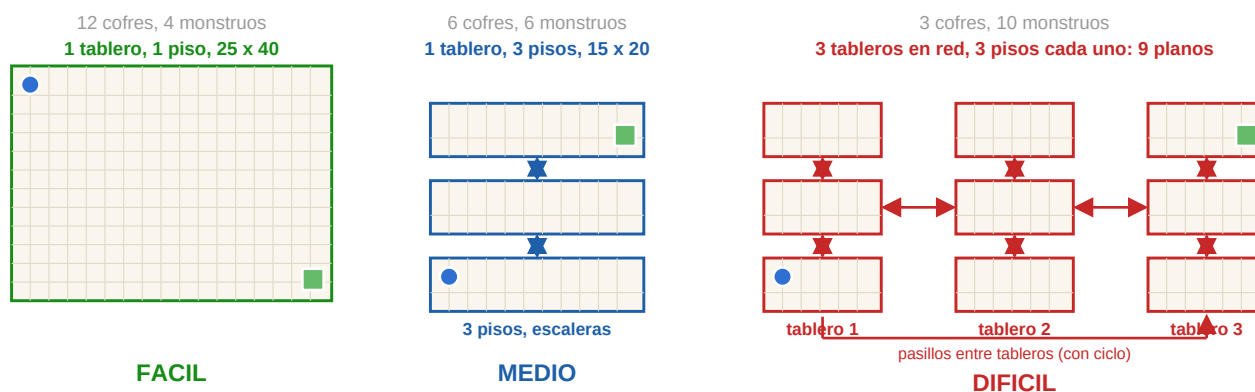


Figura 2. Fácil: un plano grande. Medio: un tablero de tres pisos unidos por escaleras. Difícil: tres tableros de tres pisos, unidos entre sí por pasillos que forman un ciclo. El punto azul es el inicio; el cuadrado verde, la salida.

	Fácil	Medio	Difícil
Tableros	1	1	3, en red
Pisos por tablero	1	3	3
Planos en total	1, grande (25 × 40)	3 (15 × 20)	9 (15 × 20)
Conexiones	ninguna	escaleras entre pisos	escaleras entre pisos y pasillos entre tableros
Cofres	muchos (12)	6	pocos (3)
Monstruos	4	6	10

	Fácil	Medio	Difícil
La salida	en el único plano	en el piso 3	en cualquier plano

Una escalera y un pasillo son lo mismo para el modelo: una Conexión de una celda de un plano a una celda de otro. Lo que cambia es la geometría. La escalera lleva a la **misma posición** del piso de arriba o de abajo. El pasillo está en el **borde** de un tablero y lleva al borde del tablero vecino. En Difícil, los tres tableros forman un **grafo** que puede tener ciclos, y cada tablero es a su vez una pila de tres pisos: nueve planos para recorrer, y una sola salida.

Configuración y generación

El archivo de configuración

configuracion.json se lee al iniciar. Si falta, se rechaza con un mensaje claro: no hay valores escondidos en el código.

```
{
  "energiaInicial": 100,
  "costos": { "piso": 1, "agua": 3, "lava": 8 },
  "semilla": null,
  "dificultades": {
    "FACIL": { "tableros": 1, "pisos": 1, "filas": 25, "columnas": 40, "cofres": 12, "monstruos": 4 },
    "MEDIO": { "tableros": 1, "pisos": 3, "filas": 15, "columnas": 20, "cofres": 6, "monstruos": 6 },
    "DIFICIL": { "tableros": 3, "pisos": 3, "filas": 15, "columnas": 20, "cofres": 3, "monstruos": 10 }
  }
}
```

La **semilla** manda: con null cada partida es distinta; con un número, la misma semilla genera exactamente el mismo mundo. Así se puede reproducir un bug, escribir un test, y jugar todos el mismo nivel.

El generador de laberintos: la interfaz que se intercambia

Es la **única pieza** que se intercambia con otro grupo, y por eso su contrato está reducido a lo mínimo: arreglos y caracteres, ninguna clase que haya que compartir.

```
public interface GeneradorDeLaberinto {
  /**
   * Genera un plano de filas x columnas: '#' pared, '.' piso.
   * Con la misma semilla devuelve siempre el mismo plano.
   */
  char[][] generar(int filas, int columnas, long semilla);
}
```

Un plano es **válido** si cumple cuatro condiciones, que la cátedra verifica con su validador y el grupo tiene que verificar con sus tests:

- 1) **Bordes de pared**: toda la primera y última fila, y la primera y última columna, son #.
- 2) **Conexo**: desde cualquier celda de piso se llega a cualquier otra (recorrido en anchura con cola).
- 3) **Con bifurcaciones**: al menos el 10 % de las celdas de piso tienen **tres o más vecinos** de piso. Un laberinto que es un solo pasillo serpenteante no sirve: tiene que haber caminos que se abren.
- 4) **Con proporción**: entre el 30 % y el 60 % de las celdas son piso.

El algoritmo recomendado es el de **backtracking con pila**:

- 1) Todas las celdas empiezan como pared. Se elige una celda de piso inicial y se apila.
- 2) Mientras la pila no esté vacía: se mira la celda del tope; se eligen al azar sus vecinas a distancia 2 que todavía sean pared; si hay alguna, se abre el camino hasta ella (la celda de en medio pasa a piso) y se apila; si no hay ninguna, se desapila.
- 3) Cuando la pila queda vacía el laberinto es conexo por construcción. Para las **bifurcaciones**, se abren después algunas paredes elegidas al azar entre dos celdas de piso: cada una crea un ciclo y un camino

alternativo.

La pila tiene que ser **la del grupo**, no la recursión de Java: en un plano de 25×40 la recursión llega a 500 niveles, y la pregunta 5 del cuestionario pide explicar por qué eso importa.

Lo que el generador no hace

El generador entrega paredes y piso. Todo lo demás lo hace el juego del grupo, **sobre cualquier plano válido**, propio o ajeno:

- convertir un porcentaje de piso en **agua** o **lava**;
- ubicar al azar los **cofres** y los **monstruos** en piso libre;
- elegir el **inicio** al azar, y poner la **salida** en la celda de piso más lejana al inicio, medida con BFS con cola: así el camino nunca es trivial;
- ubicar las **escaleras** (misma posición en los dos pisos) y los **pasillos** (en los bordes), y verificar con el grafo del mundo que la salida es **alcanzable** desde el inicio atravesando conexiones.

El intercambio

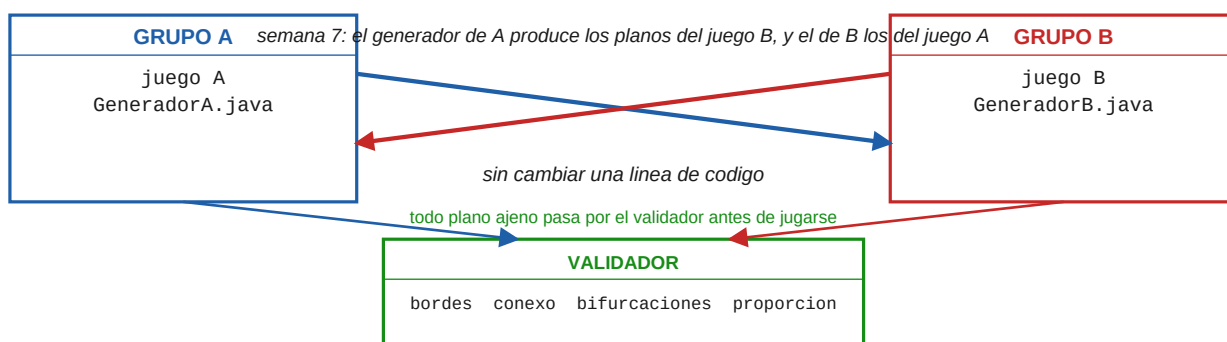


Figura 3. En la semana 7 cada grupo enchufa el generador de otro grupo en su juego. Todo plano ajeno pasa por el validador antes de jugarse.

En la **semana 7**, cada grupo publica su clase que implementa `GeneradorDeLaberinto` (un `.java` o un `.jar`) y recibe la de **otro grupo**. La enchufa en su juego, **sin cambiar nada**, y juega una partida en cada dificultad con planos generados por el generador ajeno. Antes de jugar, la pasa por su validador: si el plano ajeno no cumple las cuatro condiciones, el informe dice cuál falla.

Partida guardada, ranking y records

`guardar()` escribe un JSON con **todo** el estado (regla 10) y `cargar()` lo reconstruye. `ranking.json` guarda nombre, puntaje, dificultad y fecha, y se carga al iniciar en el ABB. Todo lo que es un **valor** va en un record: Posicion, Ubicacion (tablero, piso, posición), Conexion, EventoProgramado, Puntaje, EntradaDeRanking y la configuración leída. Para JSON se puede usar **Gson** o **Jackson**: es la única librería externa permitida en el modelo. `java.util.Random` está permitido para el generador.

La salida gráfica: BMP obligatorio, ventana opcional

Lo obligatorio

La acción del héroe se lee por **consola** (`w` a `s` d para moverse, `u` para usar, `e` para esperar, `g` para guardar, `q` para salir). Después de **cada paso**, el juego escribe el plano actual en un archivo **BMP**: `salida/paso-0001.bmp`, `paso-0002.bmp`, y así. Cada celda es un cuadrado de N píxeles con un color por símbolo; el héroe y los monstruos, un cuadrado o un círculo de otro color encima. Abajo o al costado, con rectángulos, la energía. Al terminar, los archivos son la **película de la partida**.

Se hace con `BufferedImage` e `ImageIO.write(imagen, "bmp", archivo)`, que son parte del JDK: no es una librería externa. El BMP no se comprime, y por eso la pregunta 6 del cuestionario pide calcular cuánto pesa.

Lo opcional, que suma puntos

La cátedra provee `JuegoVisual` (una ventana con el tablero, sprites y teclado) y `ReproductorDeSonido` (efectos y música en MP3), en librerías/. Están pensadas para reemplazar la vista BMP **en unas pocas líneas**: donde la vista obligatoria escribe un archivo, la gráfica llama a `setCelda` y `moverPersonaje`; donde lee un char de la consola, llama a `esperarTecla()`. Con `leerTecla()`, que no bloquea, los monstruos pueden moverse por tiempo.

Si el grupo quiere **subir el nivel**, es por acá, y vale hasta **10 puntos de bonus**. Pero **primero tiene que andar con BMP**. Para que el reemplazo sea de verdad de pocas líneas, hay una interfaz `Vista` en el diseño del grupo, con `VistaBmp` obligatoria y `VistaGrafica` opcional. La pregunta 8 del cuestionario pregunta qué hubo que tocar en el modelo para agregar la gráfica: la respuesta correcta es *nada*.

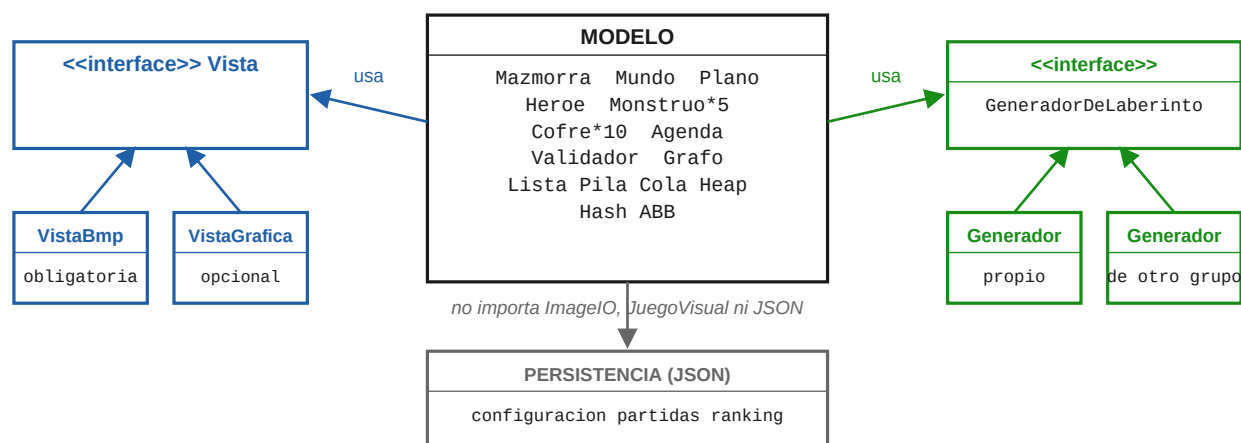


Figura 4. El modelo en el centro, sin saber quién lo dibuja, quién genera los planos ni cómo se guarda. Las dos interfaces que lo rodean son las que hacen intercambiable cada pieza.

Monstruos y cofres: el polimorfismo del grupo

Cinco tipos de monstruo

Monstruo es abstracta. Cada tipo redefine **cómo decide moverse** y **qué pasa al alcanzar al héroe**. La cátedra define dos; el grupo agrega hasta llegar a **cinco**:

```

public abstract class Monstruo {
    protected abstract Posicion decidirMovimiento(Plano plano, Posicion heroe);
    protected abstract void alAlcanzarAlHeroe(Heroe heroe, Agenda agenda);
}
  
```

Tipo	Se mueve	Al alcanzar al héroe	Lo define
ERRANTE	al azar	-30 de energía, vuelve a su posición inicial	cátedra
CAZADOR	una celda por el camino más corto (Dijkstra)	ídem	cátedra
...	...	...	el grupo, 3 más

Ideas, no obligatorias: uno que **duerme** hasta que el héroe se acerca a distancia 3 y ahí caza (despierta por la agenda); uno que **custodia** un cofre y sólo se mueve en un radio; uno que **roba** un objeto del inventario en vez de quitar energía; uno que **atraviesa paredes** pero avanza cada dos pasos.

Qué cuenta como tipo distinto: distinta decisión de movimiento, o distinto efecto al alcanzar. Dos monstruos que sólo difieren en cuánta energía quitan son **el mismo tipo** con distinto número.

Diez tipos de cofre

Cofre es abstracta con `abrir(Heroe heroe, Agenda agenda)`. La cátedra define dos; el grupo agrega hasta diez:

Tipo	Qué hace	Lo define
COFRE_DE_PUNTOS	suma los 100 puntos, nada más	cátedra
ZAPATOS_DE_AGUA	va al inventario; al usarlo, durante 20 pasos el agua cuesta como piso	cátedra
...	...	el grupo, 8 más

Condiciones sobre los diez:

- Al menos **3** tienen **efecto temporal**: vencen en un paso futuro, programado en la agenda (heap).
- Al menos **2** cambian la relación del héroe con el **terreno** (agua, lava, paredes).
- Al menos **1** es una **trampa**: efecto negativo.
- Al menos **1** afecta a los **monstruos**: los duerme, los aleja, los revela.
- El resto, libre. Ideas: una brújula que muestra la dirección a la salida, una poción que devuelve energía, un mapa que marca el plano como visitado, una llave que abre un pasillo cerrado.

Qué cuenta como tipo distinto: distinto efecto. Dos pociones que curan distinta cantidad son **el mismo tipo**. Cofres y monstruos se ubican **al azar** en celdas de piso libres, y su tipo también se sortea entre los del catálogo.

Dónde va cada estructura

Esta tabla es **obligatoria**, con la implementación **propia de la materia**. `java.util` queda prohibido en el modelo.

Estructura	Uso obligatorio	Por qué esa y no otra
Pila	Generador de laberinto (backtracking)	Lo último que se abrió es lo primero que se retrocede
Cola	BFS : salida más lejana, alcanzabilidad, validación de conexidad de un plano ajeno	El BFS visita por capas: la primera vez que llega a una celda es por el camino más corto
Lista	Entidades de cada plano; log de la partida	Se recorren y modifican en cualquier posición
Cola de prioridad (heap)	Agenda de eventos y efectos temporales de los cofres; Dijkstra	Siempre se necesita <i>el próximo que vence</i> , nunca la lista entera ordenada
Tabla hash	Inventario (id -> cantidad); celdas visitadas por plano; catálogo de tipos de monstruo y cofre (nombre -> fábrica)	Búsqueda por clave en $O(1)$, decenas de veces por paso
ABB o AVL	Ranking de puntajes, recorrido en orden para el top 10	Mantiene el orden mientras se inserta
Grafo con pesos	Cada plano: celda transitable = vértice, movimiento = arista con costo. Y el mundo : plano = vértice, conexión = arista	Dijkstra de los cazadores (dentro de un plano); alcanzabilidad de la salida a través de conexiones

El plano visto como grafo

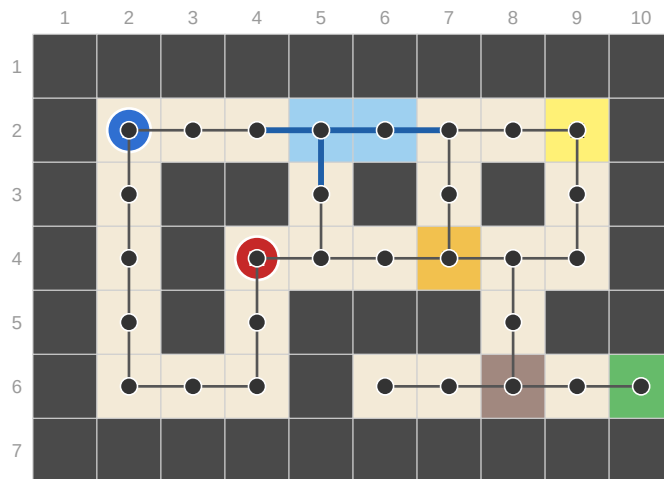


Figura 5. El plano de la figura 1 con sus vértices y aristas. Las aristas gruesas azules entran o salen del agua: cuestan 3; las demás, 1. El camino más corto de un cazador es el de menor costo total, no el de menos pasos.

Regla de oro: **el modelo no sabe cómo se dibuja ni cómo se guarda**. Ninguna clase del modelo importa ImageIO, JuegoVisual, Swing ni JSON.

Restricciones

- **Prohibido java.util en el modelo:** ArrayList, HashMap, PriorityQueue, Stack, LinkedList, TreeMap y la API de Streams. Se usan las estructuras propias. Excepciones: java.util.Random para el generador, y java.util en las vistas y en el código de JSON, que son capas de borde.
- Sin hilos, salvo el temporizador de Swing si el grupo eligió tiempo real en la vista gráfica opcional.
- Ninguna clase del modelo importa ImageIO, JuegoVisual, Swing, Gson ni Jackson.
- Toda estructura propia tiene su **test** de unidad, aparte de los tests del juego.

Organización del equipo

Seis personas y ocho semanas se desperdician sin roles. Cada grupo asigna **un responsable** por área. Responsable no es *el que lo hace solo*: es el que responde por que esté hecho y probado.

Área	Responsable de
Modelo	Mazmorra, Heroe, reglas, agenda, conexiones entre planos
Estructuras	Lista, pila, cola, heap, hash, ABB, con sus tests
Generación y grafo	Generador de laberinto, validador, BFS, alcanzabilidad, Dijkstra
Monstruos y cofres	Las jerarquías, los 5 y los 10 tipos, el catálogo, sus tests
Persistencia y vista	Configuración, guardado, ranking, VistaBmp, y la gráfica si la hacen
Integración	main, tests de punta a punta, Git, informe

Los hitos

Cada dos semanas hay una **demo de 10 minutos** frente a la cátedra. No es optativa y no se recupera. Siguen el orden en que la materia da cada estructura.

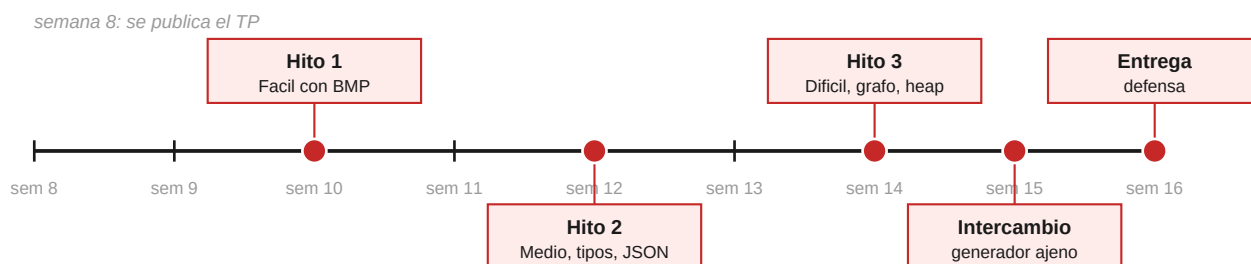


Figura 6. Las semanas son las del cuatrimestre. El TP se publica en la semana 8 y se defiende en la 16.

Sem. TP	Sem. materia	Hito	Se tiene que poder ver
2	10	Fácil con BMP	configuracion.json leído, laberinto generado con pila y semilla, validador de planos, el héroe se mueve por consola, gana y pierde, errantes, cofre de puntos, y un BMP por paso .
4	12	Medio, persistencia y tipos	Escaleras entre 3 pisos, guardar y retomar, ranking en ABB, inventario en hash, los 5 monstruos y los 10 cofres (los efectos temporales pueden faltar).
6	14	Difícil, grafo y heap	3 tableros en red de 3 pisos, salida por BFS más lejana, alcanzabilidad a través de conexiones, cazadores con Dijkstra, agenda y efectos temporales en heap.
7	15	Intercambio	El generador de otro grupo generando los planos de su juego.
8	16	Entrega y defensa	Informe, ventana gráfica y sonido si los hicieron, defensa oral.

Tests

Se entregan como mínimo:

- **Un test por estructura**, sola, con Integer o String, sin nada del juego.
- Tests de reglas: cada regla numerada de la sección *El juego* tiene al menos uno.
- **Generador y validador**: la misma semilla genera el mismo plano; todo plano generado cumple las cuatro condiciones; un plano hecho a mano que viola cada condición es rechazado por el validador, uno por uno.
- **Mundo**: la salida es alcanzable desde el inicio en las tres dificultades, atravesando escaleras y pasillos.
- **Guardar y retomar**: los dos estados son iguales, plano actual incluido.
- **Cada tipo** de monstruo y de cofre tiene un test de su comportamiento.
- **BMP**: después de un paso existe el archivo, y tiene el tamaño en píxeles que corresponde al plano.

Se corren con `./gradlew test` y tienen que pasar en verde en la entrega.

Cuestionario

Responder el siguiente cuestionario en el informe:

- 1) ¿Por qué la agenda es una cola de prioridad y no una lista ordenada? Comparen insertar y sacar el próximo en cada caso.

- 2) ¿Qué diferencia de costo hay entre buscar en el inventario si es lista y si es tabla hash? ¿Cuántas veces por paso se consulta?
- 3) ¿Por qué el ranking es un ABB y no un arreglo ordenado? ¿Qué pasa si los puntajes llegan en orden creciente, y cómo lo resuelve el AVL?
- 4) Dijkstra en un plano de 25×40 : ¿cuántos vértices y aristas tiene el grafo? ¿Cuál es el costo de un camino mínimo?
- 5) El generador usa **su** pila y no la recursión de Java. ¿Qué profundidad alcanza la recursión en un plano de 25×40 , y qué pasaría con la pila de la JVM? ¿Por qué el laberinto resultante es conexo, y por qué hay que abrir paredes después para que tenga bifurcaciones?
- 6) Un BMP de un plano de 25×40 con celdas de 16 píxeles: ¿cuántos bytes ocupa? ¿Y la película de una partida de 300 pasos? Comparen con lo que ocuparía en PNG, y expliquen la diferencia con lo que vimos entre WAV y MP3.
- 7) El generador de otro grupo corrió en su juego. ¿Pasó el validador a la primera? Si no, ¿qué condición falló y por qué la interfaz no la pudo impedir?
- 8) Si hicieron la ventana gráfica: ¿qué tuvieron que cambiar en el modelo? Si la respuesta no es *nada*, ¿qué estaba mal acoplado?

Normas de entrega

Trabajo práctico **grupal**: 6 integrantes.

Reglas generales: respetar el **Apéndice A**.

- **Repositorio Git** por grupo, con la cátedra como colaboradora desde la semana 1. El código se entrega en el repositorio: se evalúan los commits, y **cada integrante** tiene que tener commits propios en su área a lo largo de las 8 semanas, no un volcado final.
- **Informe** en PDF, dentro del repositorio y subido al campus: decisiones de diseño, diagrama de clases con las dos jerarquías, respuestas al cuestionario, manual de usuario, manual del programador, y una sección por integrante con lo que hizo. Nombre: GrupoNN-TP2 . pdf.
- **La clase del generador**, configuracion.json y una carpeta salida/ con los BMP de una partida de ejemplo, en el repositorio.
- **Defensa oral** en la semana de defensas grupales: cada integrante explica su área y responde sobre cualquier otra.

La fecha de entrega vence el día << **COMPLETAR: DD/MM/26** >> a las 23.59 hs. La defensa es en la semana 16 del cuatrimestre.

Se evaluará: funcionalidad, eficiencia, algoritmos utilizados, buenas prácticas de programación, modularización, documentación, gestión de memoria y estructuras de datos.

Rúbrica

Criterio	Peso
Las 7 estructuras usadas donde corresponde, con sus tests	15
Reglas del juego completas y verificadas por tests	10
Generador con pila y semilla, validador, configuración, tres dificultades	20
5 monstruos y 10 cofres con polimorfismo real	15
Grafo: alcanzabilidad a través de conexiones, salida por BFS, Dijkstra	10
Persistencia: guardado exacto, ranking, BMP por paso	10
Intercambio: el generador ajeno corre sin cambios, y el propio pasa el validador ajeno	10

Criterio	Peso
Hitos cumplidos en fecha	5
Informe, cuestionario y defensa	5
Ventana gráfica con JuegoVisual y sonido, sin tocar el modelo	bonus +10

La nota es **individual**: parte de la grupal y se ajusta con los commits, la defensa y la sección personal del informe.

Apéndice A

- 1) Usar las siguientes convenciones para nombrar identificadores.
 - a) **Clases**: Los nombres de clases siempre deben comenzar con la primera letra en mayúscula en cada palabra, deben ser simples y descriptivos. Se concatenan todas las palabras. Ejemplo: Coche, Vehiculo, CentralTelefonica.
 - b) **Métodos**: Deben comenzar con letra minúscula, y si está compuesta por 2 o más palabras, la primera letra de la segunda palabra debe comenzar con mayúscula. De preferencia que sean verbos. Ejemplo: arrancarCoche(), sumar().
 - c) **Variables y objetos**: las variables siguen la misma convención que los métodos. Por ejemplo: alumno, padronElectoral.
 - d) **Constantes**: Las variables constantes o finales, las cuales no cambian su valor durante todo el programa, se deben escribir en mayúsculas, concatenadas por "_". Ejemplo: ANCHO, VACIO, COLOR_BASE.
- 2) Si el trabajo práctico requiere archivos para procesar, entregar los archivos de prueba en la entrega del TP. Utilizar siempre **rutas relativas** y no absolutas.
- 3) Entregar el informe explicando el TP realizado, manual de usuario y manual del programador en un mismo PDF dentro del repositorio.
- 4) Comentar el código. Todos los tipos, métodos y funciones deberían tener sus comentarios.
- 5) Modularizar el código. No entregar 1 o 2 archivos, separar cada clase.
- 6) Si cualquier estructura de control tiene 1 línea, utilizar **{ }** siempre, por ejemplo:

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```